

W13D4 - XSS e SQLi

metasploitable - exploit

XSS & SQLi

Danilo 'danix' Macrì - mia.mail@mio.sito

Scan started on 08/10/2025 - duration (days): 1

Contents

Sommario	1
Scope	1
Tecnologie utilizzate	1
Exploit	2
XSS - cross-site scripting	2
SQLi - SQL injection	5

Sommario

Oggi eseguiremo un pentest mirato alle vulnerabilità di cross-site scripting (XSS) e SQL injection (SQLi) sulla nostra macchina Metasploitable2.

Scope

Il target dei nostri test è come sempre la macchina metasploitable

- 192.168.50.101
- http://metasploitable

Inizieremo i nostri test settando la difficoltà a low per poi proseguire verso i livelli medium e high.

Tecnologie utilizzate

Per i nostri test utilizzeremo i seguenti software:

- sqlmap 1.9.9 #stable

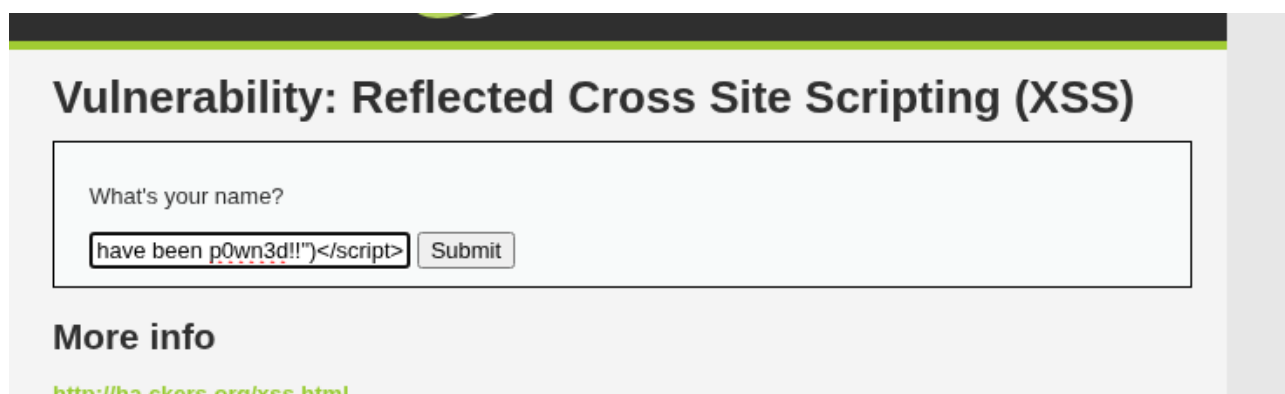
Exploit

XSS - cross-site scripting

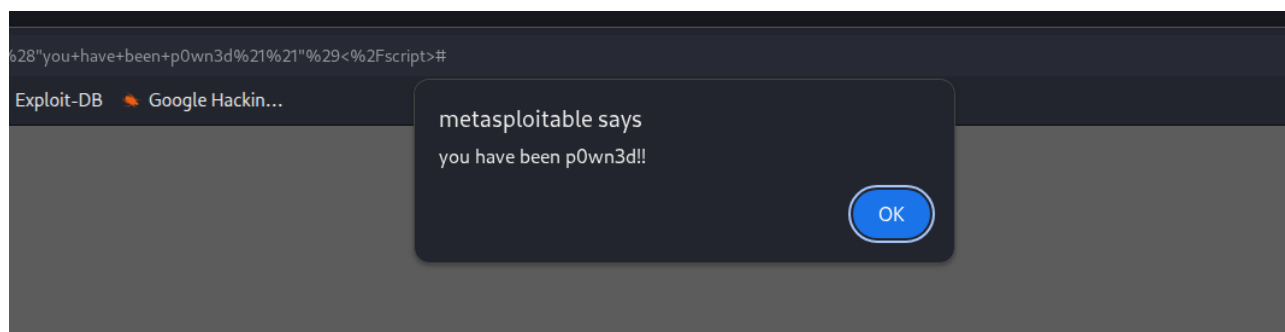
L'attacco di cross-site scripting consiste nell'andare a inserire codice malevolo in campi di input all'interno di webapp, allo scopo di alterare la funzionalità originale della webapp attaccata. Si fa leva su una cattiva implementazione dei campi di input da parte del programmatore che non ha sanificato correttamente l'input dell'utente prima di processarlo.

XSS - Security Low done Per l'exploit di cross-site scripting introdurremo il seguente codice javascript nel campo di testo della DVWA:

```
<script>window.alert("you have been p0wn3d!!")</script>
```



Una volta inserito il nostro codice, l'input viene passato alla pagina php che lo legge e lo esegue, in quanto non c'è nessun filtro che sanitizzi il testo inserito.

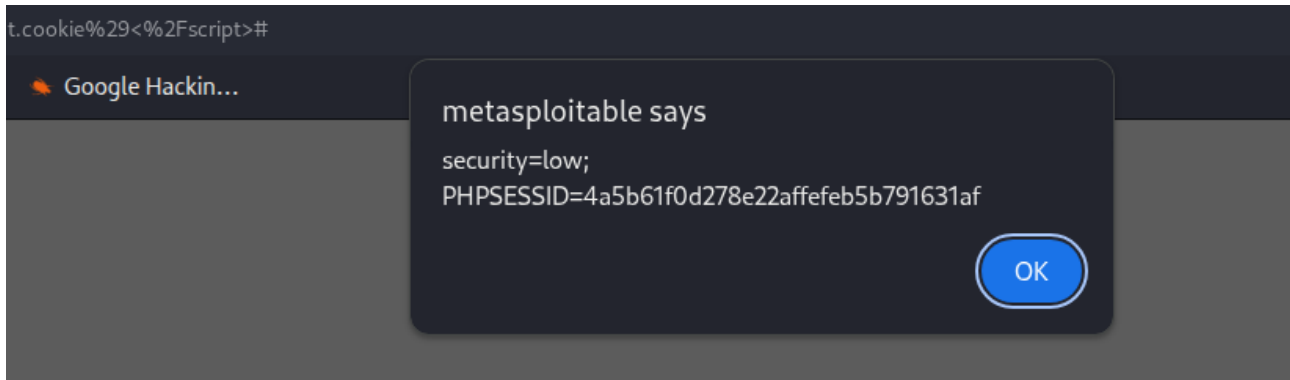


e il browser ci mostra il nostro messaggio minaccioso.

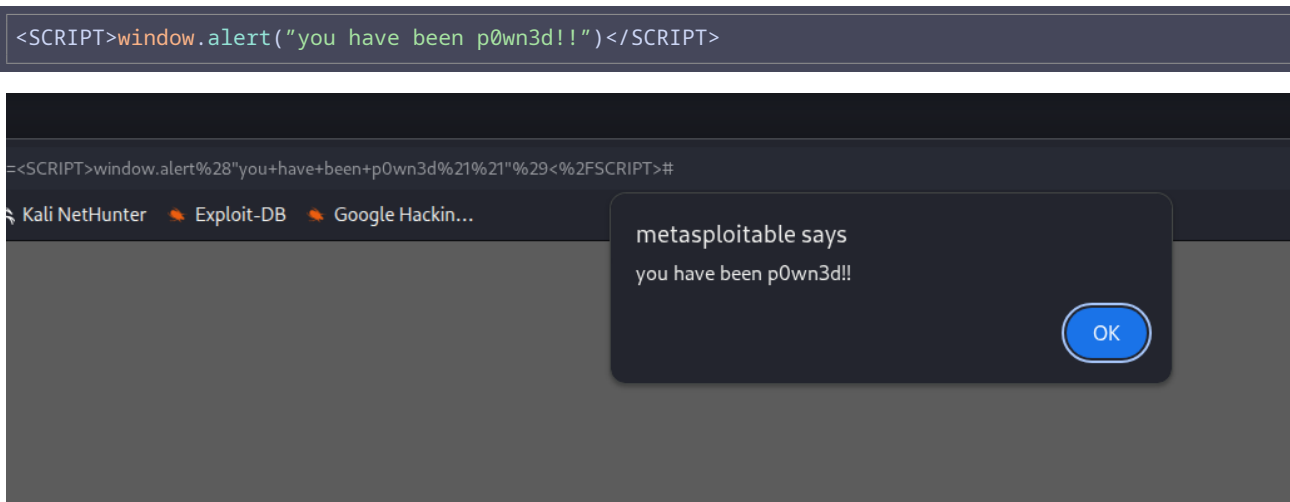
Un'altra injection (*lo strumentopolo misterioso*) che ci tornerà utile tra poco è questa:

```
<script>window.alert(document.cookie)</script>
```

che ci restituirà il cookie di sessione attivo



XSS - Security Medium done Aumentando la sicurezza della DVWA a medium, la pagina filtra e rimuove il tag html `<script>`, quindi non possiamo più utilizzarlo, ma andando a guardare il codice noteremo che il filtro è case-sensitive, quindi se utilizziamo il tag `<SCRIPT>` con tutte lettere maiuscole, dovrebbe passare ugualmente.



Come vediamo dallo screenshot, il filtro funziona solo se il tag è scritto in minuscolo, che è la direttiva standard di html5, ma siccome il browser deve saper leggere anche altri dialetti (*come HTML4 Transitional che è molto permissivo in termini di sintassi*) e deve riuscire a rendere ugualmente le pagine anche se mal formattate, riesce a leggere tranquillamente il nostro codice.

XSS - Security High impossible La situazione cambia drasticamente aumentando il livello a high, in questo caso la pagina utilizza la funzione php `htmlspecialchars()` che serve a sanificare qualsiasi simbolo utilizzato in html, traducendolo nella sua safe entity.

Reflected XSS Source

```
<?php
if(!array_key_exists ("name", $_GET) || $_GET['name'] == NULL || $_GET['name'] == ''){
    $isempty = true;
} else {
    echo '<pre>';
    echo 'Hello ' . htmlspecialchars($_GET['name']);
    echo '</pre>';
}
?>
```

Se ad esempio proviamo a passare `<script>` questo verrà tradotto in simboli html safe come `<script>` in modo da impedirne l'interpretazione da parte del browser. Questo è il metodo più efficace per sanificare l'input dell'utente, rendendo di fatto sicura la pagina.

SQLi - SQL injection

Come per gli attacchi XSS, anche per l'SQL injection, si andrà a inserire del codice malevolo (payload) all'interno di campi di testo che vadano a interrogare un database, allo scopo di attraversare il DB e riuscire ad ottenere informazioni utili.

In questo genere di attacchi la prima cosa da fare è identificare un injection point, un campo di testo non propriamente sanificato che permetta l'interruzione del normale funzionamento della pagina web su cui si trova. Per farlo potrebbe bastare inserire un singolo carattere `'` all'interno del campo.

Questa è una normale query SQL:

```
SELECT name, city FROM db.table WHERE id='$id';
```

Come vediamo, al campo id viene passata una variabile, se quella variabile contiene un numero la query verrà eseguita, ma se noi passiamo un ulteriore apice in quella variabile, la query risulterà sbagliata, e l'SQL engine potrebbe restituirci un errore, rivelandoci un injection point. Ne vediamo un esempio nella dvwa:

vulnerabilities/sqli/?id=%27&Submit=Submit#

Kali Forums Kali NetHunter Exploit-DB Google Hackin...

that corresponds to your MySQL server version for the right syntax to use near '''' at line 1

Una volta trovata la vulnerabilità possiamo utilizzare un tool come **sqlmap** per ottenere il dump del database e andare a cercare informazioni interessanti all'interno del database, come nomi utente e password.

Per usare efficacemente sqlmap all'interno della dvwa avremo bisogno di estrapolare i cookie di sessione, utilizzando *lo strumentopolo misterioso* visto nell'esercizio precedente di cross-site scripting, andremo quindi a salvare il nostro cookie in una variabile in bash

```
(kali@kali)-[~]  
$ c="security=low; PHPSESSID=4a5b61f0d278e22affefeb5b791631af"
```

e andiamo adesso a recuperare l'url della pagina da attaccare, lo passeremo come argomento a sqlmap dopo averlo a sua volta salvato in una variabile

```
(kali@kali)-[~]  
$ u="http://metasploitable/dvwa/vulnerabilities/sqli/?id=asdf&Submit=Submit#"
```

il nostro comando sarà quindi simile a questo:

```
sqlmap -u $u --cookie=$c --data="id=1" --dbms="mysql"
```

abbiamo passato i seguenti parametri:

- u** l'url da attaccare, preso direttamente dalla pagina web che stavamo visitando
- cookie** il cookie di sessione, in questo caso estratto nell'esercizio precedente
- data** il parametro della query vulnerabile al nostro attacco
- dbms** il database engine utilizzato, è un parametro opzionale e se non lo specifichiamo, sqlmap è in grado di riconoscere e testare da solo i vari engine, ma siccome noi lo abbiamo estrapolato dal nostro test iniziale, ha senso passarlo per restringere i test.

asdf

